

# Data Science

---

## Lecture 08: Cluster Analysis

Prof. Dr. Michael Granitzer

# Where We Left Off

Lecture 07 ended with a representation: typed, cleaned, scaled attributes. Today we ask that data for groups, with no label to check against — so the only definition of "group" is the one we hand the algorithm.

## Lectures 04–06

labels to learn from



## Today

no labels

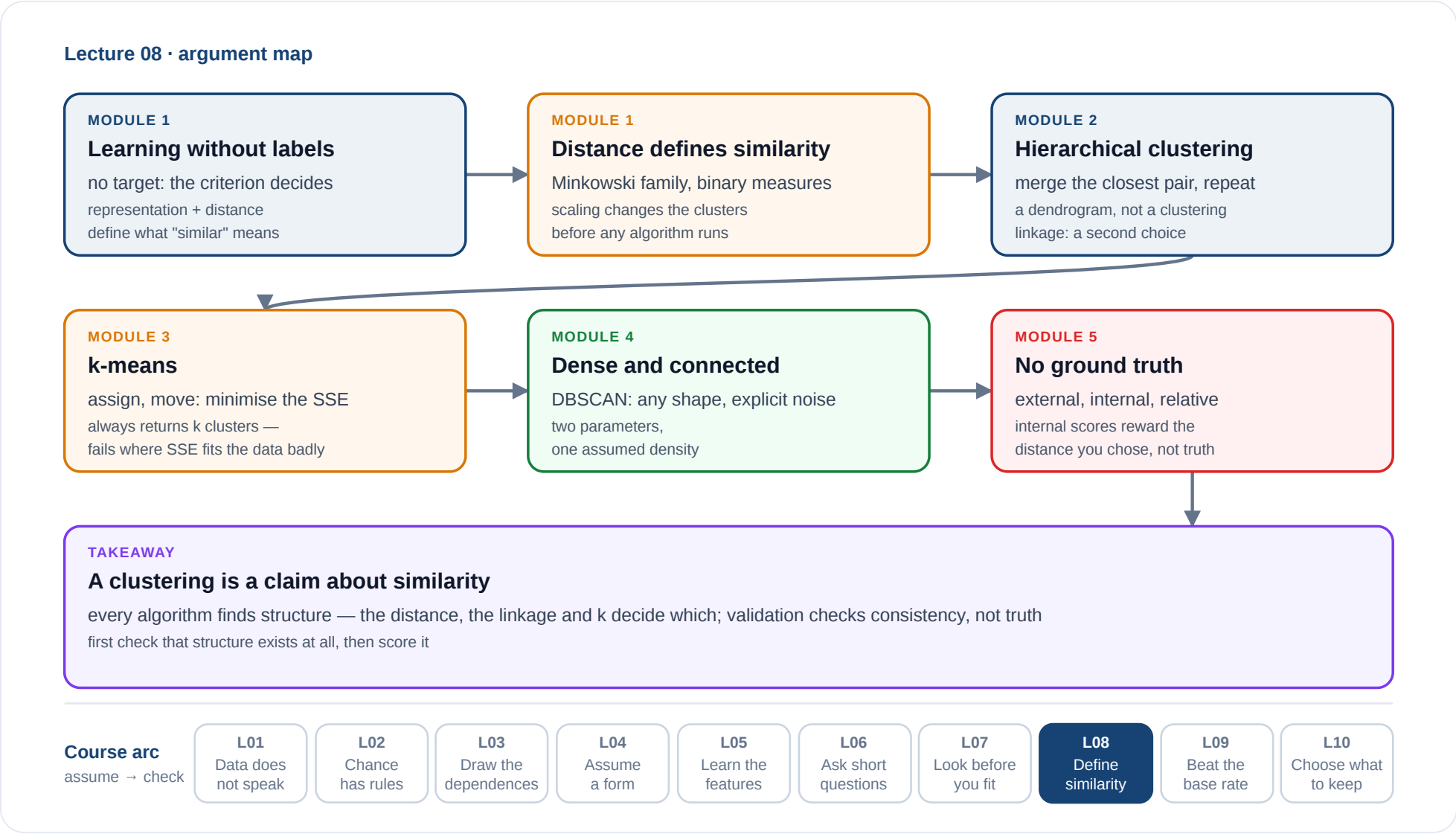


## The assumption

a distance, and a shape of cluster

Precision and recall from Lecture 04 return in Module 5 — but only where labels happen to exist.

# Argument Map



# Module 1

---

## Learning Without Labels

**Guiding Question:** *With no target concept, what makes one grouping of the data better than another?*

# Learning Without a Target

## Supervised

A **target concept** is given: each example carries a label, and the objective is to reproduce it on new data.

## Unsupervised

**No target concept.** The data carries no labels, and the objective must come from the structure itself.

**Cluster analysis** is the unsupervised classification of a set of objects into groups such that objects **within** a group are similar and objects in **different** groups are dissimilar.

# The Stages of a Cluster Analysis

## Represent

Choose the attributes and their scaling.

## Measure

Choose a distance or similarity.

## Group

Run an algorithm that optimises some criterion.

## Interpret

Decide whether the groups mean anything.

Every clustering algorithm **always returns clusters**, including on data that has none. The output is never evidence that structure exists.

Clustering has no ground truth. The **representation and the distance encode what "similar" means** — the algorithm only optimises the criterion you handed it.

# What a Distance Must Satisfy

A **metric**  $d$  satisfies, for all  $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$ :

1.  $d(\mathbf{x}_1, \mathbf{x}_2) \geq 0$  — non-negativity
2.  $d(\mathbf{x}_1, \mathbf{x}_2) = 0 \Leftrightarrow \mathbf{x}_1 = \mathbf{x}_2$  — identity
3.  $d(\mathbf{x}_1, \mathbf{x}_2) = d(\mathbf{x}_2, \mathbf{x}_1)$  — symmetry
4.  $d(\mathbf{x}_1, \mathbf{x}_3) \leq d(\mathbf{x}_1, \mathbf{x}_2) + d(\mathbf{x}_2, \mathbf{x}_3)$  — triangle inequality

The pairwise distances of  $n$  objects form the **distance matrix**, which is what most clustering algorithms actually consume.

# Numeric Attributes: the Minkowski Family

$$d(\mathbf{x}_1, \mathbf{x}_2) = \left( \sum_{i=1}^p |x_{1i} - x_{2i}|^r \right)^{1/r}$$

$$r = 1$$

**Manhattan** — sum of absolute differences. Robust to single large deviations.

$$r = 2$$

**Euclidean** — straight-line distance. The default.

$$r \rightarrow \infty$$

**Chebyshev** — the largest single-attribute difference dominates.

Minkowski distances are **not scale-invariant**. An attribute measured in metres and one in millimetres do not contribute equally — standardise first, or the unit of measurement chooses your clusters.

# Binary Attributes

With  $f_{11}$  the count of attributes present in both objects,  $f_{00}$  absent in both, and  $f_{01}, f_{10}$  present in exactly one:

## Simple Matching Coefficient

$$\text{SMC} = \frac{f_{11} + f_{00}}{f_{11} + f_{00} + f_{01} + f_{10}}$$

Counts **shared absences** as agreement.

## Jaccard Coefficient

$$J = \frac{f_{11}}{f_{11} + f_{01} + f_{10}}$$

**Ignores** shared absences.

For sparse data the difference is decisive: two shopping baskets agree on thousands of products **neither** contains. SMC would call them nearly identical; Jaccard would not.

# Mini-Exercise

Two documents over a 10,000-word vocabulary: A contains 30 words, B 35, and they share 20. SMC comes out at 0.9975. What is Jaccard — and which of the two is the honest similarity?

The distance measure **is** the definition of similarity. Scale sensitivity and the treatment of shared absences change the clustering before any algorithm runs.

# Mini-Exercise

Two documents over a 10,000-word vocabulary: A contains 30 words, B 35, and they share 20. SMC comes out at 0.9975. What is Jaccard — and which of the two is the honest similarity?

$J = \frac{20}{20+10+15} = \frac{20}{45} \approx 0.44$ . Jaccard is honest: SMC's 99.75% comes almost entirely from the 9,955 words **neither** document uses.

The distance measure **is** the definition of similarity. Scale sensitivity and the treatment of shared absences change the clustering before any algorithm runs.

# The Churn Shop: Segmenting Customers

|          | age (years) | spend (€/year) |
|----------|-------------|----------------|
| <b>A</b> | 25          | 1,000          |
| <b>B</b> | 65          | 1,050          |
| <b>C</b> | 26          | 1,400          |

# The Churn Shop: Segmenting Customers

|          | age (years) | spend (€/year) |
|----------|-------------|----------------|
| <b>A</b> | 25          | 1,000          |
| <b>B</b> | 65          | 1,050          |
| <b>C</b> | 26          | 1,400          |

## RAW UNITS

$d(A, B) \approx 64$ ,  $d(A, C) \approx 400$ : **A groups with B**, the 65-year-old. Spend decides.

## SCALED: AGE $\div$ 15, SPEND $\div$ 1,000

$d(A, B) \approx 2.7$ ,  $d(A, C) \approx 0.4$ : **A groups with C**. Age counts.

# The Churn Shop: Segmenting Customers

|          | age (years) | spend (€/year) |
|----------|-------------|----------------|
| <b>A</b> | 25          | 1,000          |
| <b>B</b> | 65          | 1,050          |
| <b>C</b> | 26          | 1,400          |

## RAW UNITS

$d(A, B) \approx 64$ ,  $d(A, C) \approx 400$ : **A groups with B**, the 65-year-old. Spend decides.

## SCALED: AGE $\div$ 15, SPEND $\div$ 1,000

$d(A, B) \approx 2.7$ ,  $d(A, C) \approx 0.4$ : **A groups with C**. Age counts.

Same customers, opposite segments: the units were a choice.

# Three Families

## Hierarchical

Build a **tree** of nested groupings, by merging (agglomerative) or splitting (divisive). No  $k$  required in advance.

## Partitional / exemplar-based

Split the data into  $k$  groups, each summarised by a representative, optimising a criterion. Requires  $k$ .

## Density-based

Clusters are **dense regions** separated by sparse ones. Finds arbitrary shapes and marks noise; requires density parameters instead of  $k$ .

No family is best. They differ in what they assume a cluster *is* — nested, compact-around-a-centre, or dense — and that assumption is the inductive bias.

Choosing an algorithm family means choosing a **definition of cluster**. The question is never "which is best" but "which shape of structure am I claiming exists".

# Module 2

---

## Hierarchical Clustering

*Guiding Question: Can we avoid committing to a number of clusters — and what does that cost?*

# The Algorithm

## HAC

Start with every object as its own cluster:  $\mathcal{C} = \{\{x_1\}, \dots, \{x_n\}\}$ .

**while**  $|\mathcal{C}| > 1$ : find the closest pair  $\{C, C'\} = \arg \min_{C_i \neq C_j} d_{\mathcal{C}}(C_i, C_j)$  replace them by their union  $C \cup C'$

The merge order is recorded as a **dendrogram** — a tree whose height shows the distance at which each merge happened. Cutting it at a chosen height yields a clustering.

No  $k$  is needed to run the algorithm — only to **read** the result.

# The Linkage Criterion

The object distance  $d$  is given;  $d_c$  — the distance between **clusters** — must still be defined, and it changes everything.

## Single link

distance of the **closest** pair.

Finds elongated shapes prone to chaining.

## Complete link

distance of the **farthest** pair.

Prefers compact, roughly equal-sized clusters.

## Group average

**mean** distance over all cross-pairs.

A compromise between the two extremes.

## Ward

merge that increases **within-cluster variance** least.

Compact, similarly-sized clusters; the common default.

# Mini-Exercise

Four points on a line at 0, 1, 5, 6. The first merges give  $\{0, 1\}$  and  $\{5, 6\}$  at height 1. At what height do these two clusters merge under **single link**, and under **complete link**?

# Mini-Exercise

Four points on a line at 0, 1, 5, 6. The first merges give  $\{0, 1\}$  and  $\{5, 6\}$  at height 1. At what height do these two clusters merge under **single link**, and under **complete link**?

Single link:  $d(1, 5) = 4$ , the closest cross-pair. Complete link:  $d(0, 6) = 6$ , the farthest. Same clusters, different heights: the linkage rescales the dendrogram and so moves where a cut lands.

# Single Link's Characteristic Failure

Single link merges on the **closest pair only**, so a thin bridge of points between two dense groups joins them: each point is close to the next, and the chain propagates.

## The same property, useful

Elongated and curved clusters are found: single link follows the shape of the data instead of imposing a compact one.

## The same property, harmful

A few noise points between two well-separated groups merge them into one long cluster nobody wanted.

One property seen from two sides, not a bug: single link is **transitive** — near-of-near counts as near. This is the **chaining effect**.

# Merges Are Irrevocable

HAC is **greedy**: it takes the locally closest pair and never reconsiders. Two objects merged early can never be separated later.

## The consequence

The dendrogram is **nested by construction**, whether or not the data is. An early mistake is inherited by every later level.

## The comparison

ID3's greedy splitting, in the opposite direction: one grows a tree top-down, the other bottom-up, and neither backtracks.

Centroid linkage can even produce **inversions** — a later merge at a *lower* height. Single, complete, average and Ward linkage are monotonic.

# What HAC Gives You

## Strengths

No  $k$  required; a **whole family** of clusterings in one dendrogram; runs on a distance matrix alone, so objects need no vector representation.

## Weaknesses

$O(n^2)$  memory for the distance matrix and  $O(n^2 \log n)$  time or worse, so it does not scale. Greedy, irrevocable merges. Strong dependence on the linkage.

HAC returns a **dendrogram**, not a clustering, and the **linkage** is a modelling decision as consequential as the distance. It trades **scalability for insight**: greedy and  $O(n^2)$ , but it shows the whole hierarchy without a  $k$ .

# Module 3

---

## k-Means

**Guiding Question:** *k-means always returns  $k$  clusters. When is that an answer, and when is it an artefact?*

# The Idea

Represent each cluster by one **exemplar** — a centroid — and assign every object to the nearest one. The quality criterion is the total squared distance to the assigned exemplar:

$$\text{SSE} = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} d(\mathbf{x}, \boldsymbol{\mu}_j)^2$$

Minimising SSE over all assignments is **NP-hard**. k-means is a heuristic that finds a local optimum — which is why its result depends on where it starts.

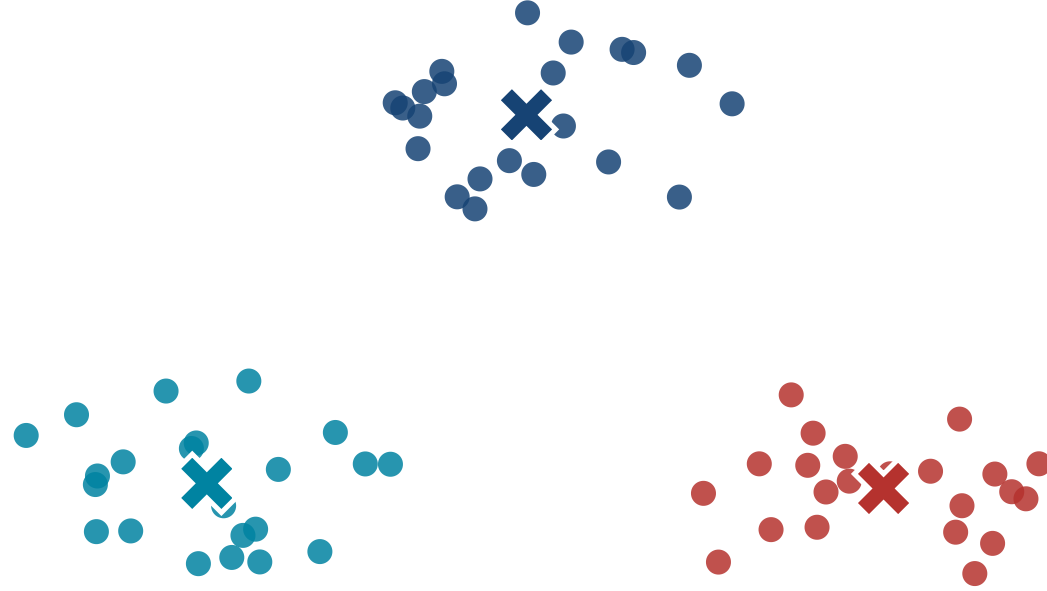
# The Algorithm

## k-means (Lloyd)

1. Choose  $k$  initial centroids.
2. **Assign** each object to the nearest centroid.
3. **Move** each centroid to the mean of its assigned objects.
4. Repeat 2–3 until the assignment stops changing.



Move each centroid to the mean of its points (round 3)



# Properties

## Fast

$O(nkp)$  per iteration, and few iterations in practice — it never needs the full distance matrix.

## Converges

SSE never increases, and there are finitely many assignments, so the procedure terminates.

## To a local optimum

The result depends on initialisation. Standard practice: restart several times and keep the best SSE.

**k-medoids** uses an actual data point as the exemplar instead of the mean. It is slower but works with any distance matrix — including where a "mean" is undefined.

# Mini-Exercise

k-means moves each centroid to the **mean** of its objects. Why the mean — and what would change with the **median**?

# Mini-Exercise

k-means moves each centroid to the **mean** of its objects. Why the mean — and what would change with the **median**?

The mean is exactly the point minimising the **sum of squared** distances to a set, so the move step solves its half of the SSE objective optimally. The median minimises the sum of **absolute** distances: you would be optimising a different, more outlier-robust criterion (k-medians).

# Takeaway

k-means alternates **assign** and **move**, minimising SSE to a local optimum. The mean is not a convenience — it is the minimiser of the criterion, which is why changing the criterion changes the update.

# Four Failure Modes

## Non-globular shapes

Concentric rings, crescents, elongated bands. SSE rewards **compact, spherical** groups, so k-means cuts such shapes across the middle.

## Differing sizes

A large sparse cluster next to a small dense one gets split, because splitting the large one lowers SSE more than respecting the boundary.

## Differing densities

The criterion counts squared distance, not density — so a diffuse cluster is torn apart to reduce it.

## Outliers

The mean is not robust: a single far point drags its centroid away from the group it is supposed to represent.

# What to Do About It

## Change the representation

Scale or transform features so the clusters *become* globular.

## Change the algorithm

Density-based methods for arbitrary shapes; k-medoids for outlier robustness.

## Change $k$

Over-clustering and then merging can recover a non-globular shape from several spherical pieces.

k-means fails exactly where its criterion stops matching the data. **Diagnose by plotting** — the failures are obvious visually and invisible in the SSE.

# Module 4

---

## Density-Based Clustering

**Guiding Question:** *Some points belong to no group at all. Which algorithms are even allowed to say so?*

# A Different Definition of Cluster

A cluster is a **dense region** of points, separated from other dense regions by sparser ones. Density is defined by two parameters:

$\epsilon$

the radius of a neighbourhood

*minPts*

how many points that neighbourhood must contain to count as dense

## Core point

Has at least *minPts* points within  $\epsilon$  — it is inside a dense region.

## Border point

Within  $\epsilon$  of a core point, but not itself dense.

## Noise point

Neither — and DBSCAN **leaves it unassigned.**

# The Algorithm

1. Label every point as core, border or noise.
2. Connect core points that lie within  $\varepsilon$  of each other — each connected group becomes a cluster.
3. Attach each border point to a cluster of an adjacent core point.
4. Leave noise points unassigned.

## Strengths

Finds **arbitrarily shaped** clusters;  
identifies noise explicitly;  $k$  is not required.

## Weaknesses

Two parameters that interact and are hard to set; it struggles when clusters have **very different densities**, since one  $\varepsilon$  must serve all.

# Mini-Exercise

DBSCAN returns one giant cluster plus a few noise points. You halve  $\varepsilon$  and get 40 tiny clusters and mostly noise. What went wrong in each run?

DBSCAN replaces "compact around a centre" with "**dense and connected**", which buys arbitrary shapes and an explicit noise category — at the price of two parameters and an assumption of uniform density.

# Mini-Exercise

DBSCAN returns one giant cluster plus a few noise points. You halve  $\varepsilon$  and get 40 tiny clusters and mostly noise. What went wrong in each run?

First run:  $\varepsilon$  too large — every dense region reaches the next, so all connect into one.

Second: too small — few points reach *minPts* neighbours, so nearly everything is noise.  $\varepsilon$  sets the density scale.

DBSCAN replaces "compact around a centre" with "**dense and connected**", which buys arbitrary shapes and an explicit noise category — at the price of two parameters and an assumption of uniform density.

# Module 5

---

## Evaluating a Clustering

**Guiding Question:** *There is no ground truth. How do you know whether the clusters mean anything?*

# Three Kinds of Validation

## External

Compare against **known labels**. Measures agreement with a ground truth we normally do not have.

## Internal

Score the clustering using **the data alone** — compactness and separation. No labels needed.

## Relative

Compare **several clusterings** of the same data to choose among them, e.g. across values of  $k$ .

Internal measures score the same geometric properties the algorithms optimise. A clustering that scores well on compactness may simply reflect that k-means minimised compactness — not that the groups mean anything.

# The Prior Question

Before asking *how good* a clustering is, ask whether the data has **cluster tendency** at all: uniformly random points produce clusters under every algorithm, and internal measures will happily score them.

Choose the validation type by what you have: **labels** → external, **only data** → internal (with suspicion), **several candidates** → relative. And check that structure exists before scoring it.

# Reusing Precision and Recall

Given known classes, treat each **cluster** as a prediction of a **class**:

## Recall of class $i$ in cluster $j$

$$\text{rec}(i, j) = \frac{n_{ij}}{n_i}$$

the share of class  $i$  that landed in cluster  $j$

## Precision of class $i$ in cluster $j$

$$\text{prec}(i, j) = \frac{n_{ij}}{n_j}$$

the share of cluster  $j$  that belongs to class  $i$

$$F_1(i, j) = \frac{2 \cdot \text{prec}(i, j) \cdot \text{rec}(i, j)}{\text{prec}(i, j) + \text{rec}(i, j)}$$

Each class is matched to the cluster maximising its  $F_1$ , and the overall score averages these, weighted by class size.

# Why the Matching Step Matters

A clustering has **no label for its clusters**. Any evaluation must first decide which cluster corresponds to which class — and different matching rules give different scores for the same clustering.

## Splitting a class

Two pure clusters covering one class score high precision, low recall each. The clustering is arguably *right* but the score punishes it.

## Merging two classes

One cluster containing both scores high recall, low precision.  $F_1$  punishes this too — as it should.

# Purity — The Simplest Matching

Give every cluster the label of its **majority class** and count the hits:

$$\text{purity} = \frac{1}{n} \sum_j \max_i n_{ij}$$

Purity only looks at precision. Splitting every class into many small clusters never lowers it — with  $k = n$  singletons, purity is 1. It needs the recall side, which  $F_1$  supplies.

# Mini-Exercise

100 objects, classes A and B (50 each). The clustering returns three pure clusters: [50 A], [25 B], [25 B]. What is class B's  $F_1$  — and is the clustering wrong?

# Mini-Exercise

100 objects, classes A and B (50 each). The clustering returns three pure clusters: [50 A], [25 B], [25 B]. What is class B's  $F_1$  — and is the clustering wrong?

Best cluster for B: precision 1, recall 0.5, so  $F_1 = \frac{2 \cdot 0.5}{1.5} \approx 0.67$  (purity is still 1). Not necessarily wrong: B was split, and if B has two sub-types, the clustering found structure the labels do not record.

# Mini-Exercise

100 objects, classes A and B (50 each). The clustering returns three pure clusters: [50 A], [25 B], [25 B]. What is class B's  $F_1$  — and is the clustering wrong?

Best cluster for B: precision 1, recall 0.5, so  $F_1 = \frac{2 \cdot 0.5}{1.5} \approx 0.67$  (purity is still 1). Not necessarily wrong: B was split, and if B has two sub-types, the clustering found structure the labels do not record.

External validation reuses precision and recall, plus a **matching** of clusters to classes. A low score can mean a bad clustering — or a label set coarser than the structure in the data.

# Comparing Two Matrices

## Proximity matrix

The pairwise **similarities** between objects: high for close pairs.

## Incidence matrix

The **ideal** structure implied by the clustering: 1 if two objects share a cluster, 0 otherwise.

A good clustering makes the two agree: pairs in the same cluster should be similar, pairs in different clusters not. Their **correlation** measures that agreement in one number, and good means **close to +1**.

With a **distance** matrix instead, the sign flips: good means strongly **negative** correlation (same cluster  $\leftrightarrow$  small distance).

# Reading It

## High correlation

The similarity structure and the cluster structure agree — the partition follows the geometry.

## What it does not tell you

Whether the geometry is meaningful.  
Correlation with a **self-imposed** ideal structure is not evidence of real groups.

Visual variant: sort the proximity matrix by cluster and display it. Genuine clusters appear as high-similarity blocks along the diagonal — and the absence of blocks is as informative as their presence.

Internal validation asks whether the partition **agrees with the distances**. It rewards consistency with the chosen distance measure, not correctness.

# Two Quantities

## Cohesion

How tight a cluster is — the within-cluster sum of distances (SSE is one form of it).

**Lower is better.**

## Separation

How far apart clusters are — the between-cluster distance. **Higher is better.**

Every internal measure combines these two, and they trade against each other: splitting a cluster always improves cohesion and usually harms separation.

# The Silhouette Coefficient

For a single object, with  $a$  its mean distance to its **own** cluster and  $b$  its mean distance to the **nearest other** cluster:

$$s = \frac{b - a}{\max(a, b)} \in [-1, 1]$$

| Value         | Reading                                                          |
|---------------|------------------------------------------------------------------|
| $s \approx 1$ | well inside its cluster, far from the others                     |
| $s \approx 0$ | on the boundary between two clusters                             |
| $s < 0$       | closer to another cluster than to its own — probably misassigned |

Example:  $a = 1$ ,  $b = 4$  gives  $s = 3/4 = 0.75$ ; swap them and  $s = -0.75$ .

# Reading the Silhouette

Average the silhouette over all objects for one score — but **plot the distribution** too. A good average can hide a cluster whose members are all near zero.

Unlike SSE, the average silhouette does not simply improve with  $k$ : it peaks where clusters are both tight and well separated, so it can select  $k$ .

Cohesion and separation are the two axes of every internal measure. The **silhouette** combines them per object, which is why its distribution is more informative than its mean.

# Worst Separation over Worst Cohesion

$$D = \frac{\min_{i \neq j} \delta(C_i, C_j)}{\max_k \Delta(C_k)}$$

where  $\delta$  is the distance between two clusters and  $\Delta$  the diameter of a cluster. **Higher is better.**

The numerator is the **closest** pair of clusters; the denominator is the **widest** cluster. The index is therefore governed entirely by the two worst cases.

# Consequences

## The virtue

A single badly separated pair or one sprawling cluster **cannot** be hidden by good behaviour elsewhere.

## The cost

Extremely sensitive to outliers and noise: one distant point inflates a diameter and collapses the index.

Dunn scores the **worst case**, so it is strict and fragile. Use it when no bad cluster is acceptable; prefer averaging measures when a few outliers are tolerable.

# The Problem

SSE **always** decreases as  $k$  grows — with  $k = n$  every point is its own cluster and SSE is zero. So the criterion cannot choose  $k$  by itself.

The same structure as training error against model capacity: the objective improves monotonically with complexity, so **the objective cannot select the complexity**.

# The Elbow

Plot SSE against  $k$  and look for the point where the curve **stops falling steeply** — where additional clusters buy little.

## The reasoning

Up to the elbow, each new cluster splits a genuinely heterogeneous group. Past it, new clusters merely subdivide homogeneous ones.

## The catch

The elbow is often **not sharp**, and different observers read different values. It is a heuristic, not a criterion.

# Alternatives

## Silhouette over $k$

Pick the  $k$  maximising the average silhouette — a genuine maximum rather than a bend.

## Gap statistic

Compare SSE against the SSE expected on **random data with no clusters**.

## Domain constraints

Often decisive: how many segments can the business actually act on?

**SSE alone cannot select  $k$** : it always improves with  $k$ . The elbow is a **heuristic reading** of that trade-off; criteria with a genuine optimum over  $k$  (silhouette, Dunn, gap statistic) formalise it, and the domain often overrules them all.